

TP n° 1 : Ollama et StreamLit

"Ollama est un outil open-source révolutionnaire qui permet d'exécuter des grands modèles de langage (comme Llama 3, Mistral, ou Phi-3) localement, directement sur votre propre ordinateur. Conçu pour simplifier au maximum le déploiement d'intelligences artificielles, il masque toute la complexité technique en encapsulant les modèles et leurs dépendances dans une interface logicielle légère et ultra-rapide.

Que ce soit via une simple invite de commandes dans le terminal ou à travers son API locale (qui s'intègre parfaitement avec des scripts Python ou des applications Streamlit), Ollama offre aux développeurs et aux passionnés une liberté totale. C'est la solution idéale pour concevoir des applications intelligentes de manière confidentielle, gratuite, et totalement indépendante d'une connexion internet ou de services cloud payants." (description générée par Gemini)

"Streamlit est un framework Python open-source qui permet de transformer des scripts de code en applications web interactives et élégantes en seulement quelques lignes, ce qui en fait le compagnon idéal d'Ollama pour créer des interfaces d'Intelligence Artificielle locales. Grâce à son intégration native avec la bibliothèque Python d'Ollama, Streamlit permet de connecter instantanément vos modèles de langage locaux à des composants web dynamiques (comme des zones de chat, des boutons ou des combobox).

En combinant la légèreté de Streamlit pour l'interface et la puissance d'Ollama pour la génération de texte confidentielle et gratuite, vous pouvez prototyper et déployer des applications d'IA générative (comme des chatbots avec recherche web ou des assistants documentaires) de manière incroyablement rapide, esthétique et entièrement locale." (description générée par Gemini)

Dans cette séance, vous allez déployer plusieurs LLM sur Ollama en mode terminal, puis développer un script Python utilisant StreamLit pour créer une interface Web pour votre ChatBot.

Utilisation de Ollama en mode console

- a) Commencez par télécharger l'exécutable Ollama sur votre machine : `https://ollama.com/download/windows`
- b) Lancez l'exécutable pour l'installer. Si vous êtes sur une des machines de l'école, appelez l'enseignant pour avoir les accès admin
- c) Tester que tout est bien installé en ouvrant un terminal et en tapant `ollama`
- d) Une fois que la vérification est effectuée, coupez le processus
- e) Télécharger un premier modèle à tester : `mistral`. Pour cela, tapez dans la console `ollama pull mistral`
- f) Tapez `ollama list`, vous devriez voir apparaître le LLM que vous venez de télécharger
- g) Exécutez-le avec la commande `ollama run mistral`
- h) Intéragissez avec le chat bot pour :
 - Trouver quelle est la date de ses données les plus récentes
 - L'interroger sur l'ESEO
- i) Terminez votre conversation avec la commande `/bye`

Le chatbot avec lequel vous venez d'intéragir est performant, mais ces données ne sont ni à jour ni suffisamment précises. C'est pourquoi vous allez mettre en place dans les séances suivantes un RAG et des requêtes Internet pour l'alimenter en données. Pour l'instant, vous allez créer un script Python qui exploite Ollama, cela permettra d'avoir une surcouche prête à l'emploi

Interface Web StreamLit

- a) Commencez par effectuer les installations via pip de ollama et streamlit
- b) Dans le même dossier, créez les fichiers `app.py` et `LLM.py`. Le premier vous servira à définir l'interface graphique et le second contient l'interface avec Ollama pour exécuter le ChatBot
- c) Tout en haut du fichier, ajoutez les import suivant :
 - `from LLM import *`
 - `import streamlit as st`
- d) Définissez votre fonction **main** et ajoutez son appel dans le point d'entrée du script

- e) Dans la fonction **main** définissez simplement le nom de votre StreamLit (utilisez la fonction `st.title`)
- f) Lancez votre script Python. Attention, StreamLit s'exécute un peu différemment, il faudra utiliser la commande suivante :

```
python -m streamlit run .\app.py
```

(Python lance StreamLit qui lance app.py)
- g) Une page d'adresse `http://localhost:8501` devrait s'ouvrir dans votre navigateur internet (elle ne contient pour l'instant que le titre que vous avez défini)
- h) Dans votre fonction `main` ajoutez un composant de type `st.chat_input` qui permet d'avoir une zone de texte pour que l'utilisateur envoie des messages. Ce composant prend en paramètre le texte qui est en *placeholder* et renvoie le texte tapé par l'utilisateur (que vous stockerez dans une variable)
- i) Juste après, dans une condition qui vérifie que la variable n'est pas vide, faites un `print`
- j) Sauvegardez votre script, mais ne relancez pas le processus, StreamLit se met à jour à la volée.
- k) Rechargez votre navigateur Internet (ou pas, il est possible qu'il se soit également rechargé tout seul), et tapez des messages dans la nouvelle zone de texte. Ces messages devraient apparaître dans votre terminal
- l) Dans le script `LLM.py` définissez la fonction `session_init` qui teste si `st.session_state` contient déjà un attribut `message` et si ce n'est pas le cas, l'initialise comme une liste vide.
- m) Ajoutez la fonction `get_previous_messages` qui renvoie cette liste de message si elle existe, et une liste vide sinon.
- n) Ajoutez la fonction `query` qui prend en paramètre une chaîne de caractères `user_input` (qui contient le message envoyé par l'utilisateur)
- o) Dans cette fonction, ajoutez le `user_input` dans la liste des messages de la session sous la forme d'un dictionnaire :
 - `"role" : "user"`
 - `"content" : user_input`
- p) De retour dans la fonction `main` de `app.py`, ajoutez l'initialisation de la session juste après la définition du titre
- q) Remplacez le `print` par un appel à la fonction `query`
- r) Ensuite, parcourez les messages de la session et écrivez les dans l'interface graphique. Pour cela, vous utiliserez le code suivant (où `msg` est le nom de la variable contenant le dictionnaire qui représente le message) :

```
with st.chat_message(msg["role"]) :
    st.write(msg["content"])
```
- s) Sauvegardez votre script et retournez sur votre navigateur Web
- t) Tapez du texte dans le champ prévu à cet effet, vos messages devraient apparaître dans le chat
- u) Ajoutez à la fin de la fonction `query` un message écrit par le bot (son rôle est "assistant") dont le contenu est toujours le même (c'est pour l'instant seulement un test)
- v) Sauvegardez votre script et retournez sur votre navigateur Web
- w) Tapez du texte dans le champ prévu à cet effet, vos messages devraient apparaître dans le chat avec le bot qui vous répond toujours la même chose

Le lien avec Ollama

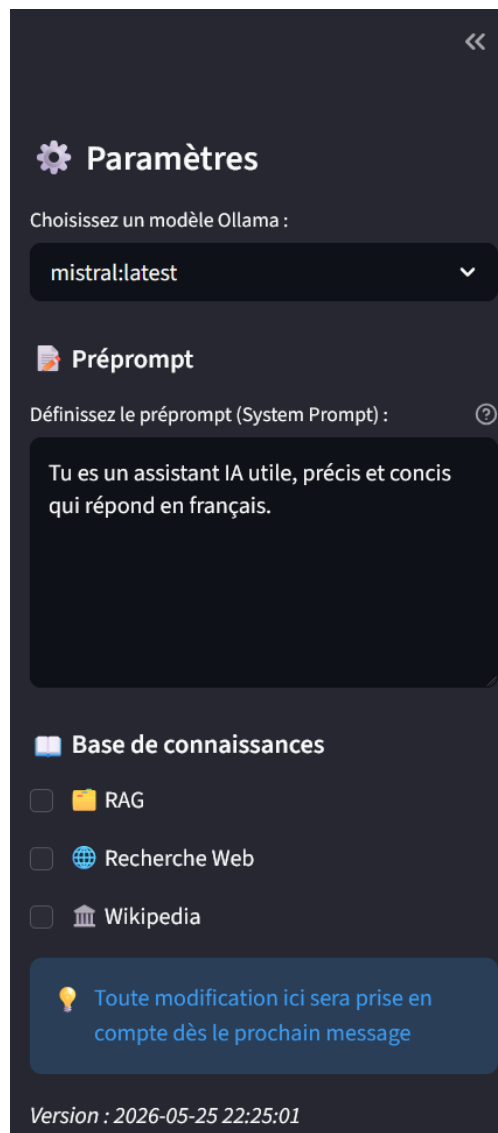
L'objectif de cette partie est de faire en sorte que le bot ne nous réponde pas par un message prédéfini, mais que le LLM stocké dans Ollama soit appelé

- a) Commencez par importer `ollama` dans votre script `LLM.py`
- b) Ouvrez un bloc de contexte (`with ...`) pour le message de l'assistant à l'aide du composant de chat dédié de Streamlit (`st.chat_message`), en lui passant la chaîne "assistant" en paramètre.
- c) À l'intérieur de ce bloc, créez un conteneur vide (`st.empty()`) nommé `response_placeholder`. Il servira à afficher le texte en streaming.
- d) Initialisez une variable textuelle vide nommée `full_response` qui servira à accumuler les morceaux de texte reçus.
- e) Appelez la méthode de chat d'Ollama (`ollama.chat`) en lui fournissant les paramètres suivants :

- `model` : mettez pour l'instant `mistral`.
 - `messages` : Une liste de deux messages : le premier a pour rôle `system` et contient le prompt système (ou préprompt) que vous définirez de manière fixe pour l'instant; le second contient l'input de l'utilisateur
 - `stream` : mettez-le à `True` pour recevoir la réponse de manière progressive.
- f) Stockez le flux de données renvoyé dans une variable `response`.
- g) Créez une boucle `for` pour parcourir chaque morceau (chunk) du flux `response`.
- h) Dans cette boucle, récupérez le contenu textuel du chunk courant et ajoutez-le à la suite de la variable `full_response`.
- i) Mettez à jour le conteneur `response_placeholder` en y écrivant (méthode `write`) la valeur actuelle de `full_response`, suivie d'un curseur visuel (ex : " | ") pour simuler une saisie en direct.
- j) Une fois la boucle terminée (lorsque la réponse est complète), videz ce qu'il reste dans `full_response` dans le conteneur `response_placeholder`.
- k) Sauvegarder la réponse de l'assistant dans l'historique de `st.session_state` avec le rôle assistant.

La barre latérale

Si vous êtes arrivé jusqu'ici, utilisez la documentation de StreamLit <https://docs.streamlit.io/> pour reproduire cette barre latérale :

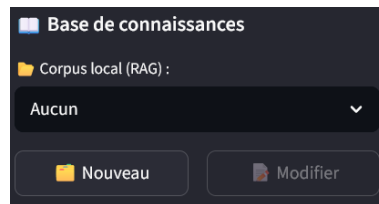


TP n° 3 : Mise en place du RAG

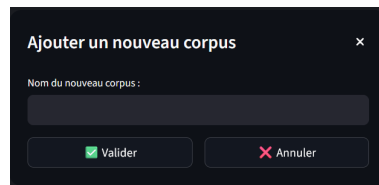
Création des corpus de documents

Dans cette première partie, nous allons gérer les corpus de documents nécessaires au fonctionnement du RAG. L'idée est de gérer tout cela depuis l'interface StreamLit pour éviter d'avoir à manipuler le gestionnaire de fichiers, qui serait source d'erreurs.

- Commencez par créer le script `corpus.py` et importer dans celui-ci le module `os` qui sert à manipuler les dossiers/fichiers en Python
- Définissez une variable globale qui indique le chemin relatif du dossier qui contiendra tous les corpus
- Faites la fonction `add_corpus` qui prend en paramètre un string et qui crée un dossier portant ce nom (s'il n'existe pas déjà)
- Faites la fonction `get_corpus_list` qui renvoie la liste des corpus (*i.e.* la liste des dossiers). Avant de renvoyer le résultat, ajouter à la première position de cette liste "Aucun" qui nous permettra de ne pas activer le RAG
- Retournez dans le script `app.py` et modifiez la barre latéral pour ajouter les composants suivants :



- Faites en sorte que la liste soit remplie automatiquement avec le nom des corpus
- Faites en sorte que le bouton "Modifier" ne soit pas actif quand le premier élément de la liste est sélectionné (on ne va pas modifier le corpus "Aucun")
- Créez la fonction `popup_ajout_corpus` qui lance un popup StreamLit. Pour cela, utilisez le décorateur `@st.dialog` avant la fonction
- Composez le popup comme ceci :



- Faites en sorte que le bouton "Valider" ajoute un nouveau corpus
- Pour les deux boutons, appelez la fonction `st.rerun` qui réexécute la page Web (et du coup fait disparaître le popup)
- Terminez en créant quelques corpus.

Intégration des documents dans les corpus

Maintenant que les corpus sont prêts, il va falloir y ajouter des documents.

- Dans le script `corpus.py`, ajoutez la fonction `add_documents_to_corpus` qui prend en paramètre le nom d'un corpus et une liste de documents
- Faites la fonction `get_documents_from_corpus` qui renvoie la liste des documents contenus dans le corpus
- Faites la fonction `delete_document_from_corpus` qui prend en paramètre le nom du corpus et le nom du document à supprimer
- Au passage, faites la fonction `delete_corpus` qui parcourt la liste des documents d'un corpus pour les supprimer, puis supprime le dossier lui-même. Il est important de procéder ainsi plutôt que de faire une suppression récursive car on aura d'autres opérations à faire dans cette boucle plus tard
- Dans le script `app.py` Créez la fonction `popup_corpus` qui est précédée par le décorateur de popup

f) Dans cette fonction, composez le popup suivant :



- g) Le tableau est un `st.data_editor` qui est rempli par la liste des documents du corpus. Pour avoir les *checkboxes*, il faut utiliser l'attribut `column_config` avec l'objet `st.column_config.CheckboxColumn`
- h) Rattachez les différentes fonctions aux boutons, et vérifiez que tout fonctionne

Mise en place de la ChromaDB

Maintenant que les corpus et les documents sont prêts, il est temps de mettre en place la base ChromaDB pour indexer leur contenu

- a) Commencez par installer dans Ollama le modèle de création des embeddings : `nomic-embed-text`
- b) Ensuite créer le script `RAG.py` et ajoutez les imports suivants :

```
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings
```
- c) Faites la fonction `ingest_documents` qui prend en paramètre le nom du corpus et une liste de documents.
- d) Dans cette fonction, commencez par créer la base ChromaDB si elle n'existe pas déjà (pour l'instant, il s'agit d'un dossier `chroma_db` qui est stocké dans le dossier du corpus)
- e) Pour chaque document de la liste reçue en paramètre :
 - Ouvrez le document avec l'objet `Document` (avec comme metadata le nom du fichier)
 - Découpez-le en chunk avec un `RecursiveCharacterTextSplitter`
 - Ajoutez les chunks à la base ChromaDB avec la méthode `from_documents`
- f) Faites la fonction `purge_document` qui prend en paramètre le nom d'un corpus et le nom du document à supprimer. Trouvez dans la base tous les chunks correspondants et supprimez-les. Ne supprimez pas le document lui-même, c'est la responsabilité du script `corpus.py`
- g) Dans le script `corpus.py`, ajoutez l'import de `RAG.py` et ajoutez ensuite un appel aux fonctions que vous venez de créer dans les fonctions correspondants
- h) Testez que toutes vos fonctions fonctionnent toujours

Ajout du RAG

Il est désormais temps de gérer l'augmentation par RAG de notre LLM

- a) Dans le script `RAG.py`, ajoutez la fonction `query_RAG` qui prend en paramètre le nom d'un corpus et l'input utilisateur
- b) Dans cette fonction, utilisez la méthode `similarity_search` pour trouver les 3 chunks les plus proches
- c) En parcourant ces chunks, construisez itérativement un contexte qui contient la source du chunk et son contenu, le tout entouré de balise pour faciliter la lecture pour le LLM

- d) Terminez la fonction en renvoyant ce contexte
- e) Dans la fonction `query` du script `LLM.py`, ajoutez un appel à la fonction que vous venez de créer
- f) Modifiez l'input de l'utilisateur en ajoutant avant sa question le contexte. Mettez des balises de contrôle du style : "en utilisant les informations suivantes : [contexte], réponds à la question : [input utilisateur]"
- g) Ajoutez des documents textes dans votre corpus, et posez des questions au LLM pour vérifier qu'il utilise bien le RAG

TP n° 4 : Les ressources Web

Wikipedia

Dans cette partie, vous allez utiliser l'API de Wikipedia pour récupérer des pages que vous injecterez ensuite dans votre corpus de documents. Cela vous permettra de répondre à des questions avec des infos fiables et "à jour"

- Commencez par créer le script `wikipedia.py` et importer dans celui-ci le module `wikipediaapi` (pensez à le pip install) et le module `io` qui sert à créer des fichiers
- Dans ce script, créez la fonction `get_wikipedia_page` qui prend en paramètre une chaîne de caractères qui contient les mots-clé pour rechercher la page Wikipedia
- Commencez par créer un objet de type `Wikipedia` et définissez vos *credentials*
- Appelez la méthode `search` de cet objet sur les mots-clés
- Récupérez le nom de la première page renvoyée et affichez-le dans la console
- Chargez cette page avec la méthode `page` et renvoyez le résultat
- Si la recherche est vide, ou si la page n'est pas trouvée, renvoyez `None`
- Testez votre nouvelle fonction avec le mot-clé "ESEO"
- Créez la fonction `create_corpus` qui prend en paramètre les mots-clés et la page Wikipedia précédemment trouvée
- Commencez par créer un nouveau corpus qui s'appelle "Wikipedia_[mot-clé]". Remplacez les espaces par des underscores
- Créez un dossier temporaire pour stocker les fichiers que vous allez créer
- Pour chaque section de la page Wikipédia, créez un nouveau fichier `.txt` qui a pour nom celui de la section (en remplaçant les espaces par des underscores) et qui contient le texte de la section
- Une fois les fichiers créés, parcourez le dossier temporaire pour ajouter vos documents au corpus. Comme la fonction que vous avez préparée précédemment prend en paramètre une liste de "UploadedFile", vous utiliserez le code suivant pour les simuler

```
for file in os.listdir("tmp"):
    file_path = os.path.join("tmp", file)
    if os.path.isfile(file_path):
        with open(file_path, "rb") as f:
            bytes_content = f.read()
            uploaded_file = io.BytesIO(bytes_content)
            uploaded_file.name = file
            print ( uploaded_file )
            add_documents_to_corpus ( corpus_name, [ uploaded_file ] )
```

- Faites la fonction `wikipedia_query` qui prend en paramètre les mots-clé pour trouver la page Wikipédia et l'input utilisateur
- Dans cette fonction, commencez par trouver la page Wikipedia
- Si la page a été trouvée, appelez la fonction `create_corpus`
- Ensuite faites une `query_RAG` qui construit un contexte à partir de la réponse de la requête et qui le renvoie
- Modifiez votre barre latérale pour ajouter les recherches Wikipedia :



- Modifiez la fonction `sidebar` pour qu'elle renvoie un dictionnaire de paramètres, dans lesquels vous ajouterez tout ce qui est paramétrable dans la barre latérale
- Passez ces paramètres à la méthode `query` du script LLM
- Dans cette méthode faites l'appel qui va bien pour ajouter les pages Wikipedia à votre ChatBot
- Testez s'il est désormais capable de répondre à vos questions sur l'ESEO



Qui est le fondateur de l'ESEO ?



Le fondateur de l'École supérieure d'électronique de l'Ouest (ESEO) est le chanoine Jean Jeanneteau.



Qui est le directeur de l'ESEO ?



Le directeur de l'École supérieure d'électronique de l'Ouest (ESEO) est Guillaume Feuvre.



Quelles sont les équipes de recherche de l'ESEO ?

Recherche Wikipedia terminée.

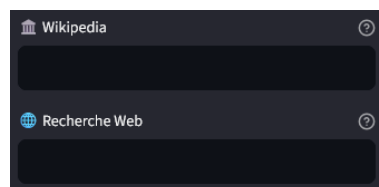


Les équipes de recherche de l'ESEO (École supérieure d'électronique de l'Ouest) sont RF-EMC (radiofréquence, antenne et RFID), GSII (contrôle non destructif, instrumentation biomédicale et capteurs optiques pour la mesure mécanique), AGE (Automatique et Génie électrique) et ÉRIS (Recherche en Informatique et Systèmes).

Les recherches Web avec DuckDuckGo

Dans cette partie, vous allez implémenter la recherche sur Internet via DuckDuckGo pour avoir accès aux informations les plus récentes

- Commencez par préparer votre barre latérale pour la recherche Internet de la même manière que vous aviez fait pour Wikipedia



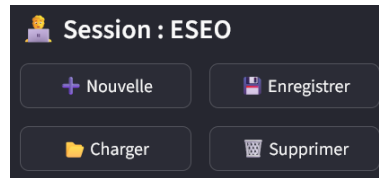
- Créez le script `Web.py` et importez les modules `ddgs` (DuckDuckGo), `httplib` (requête HTTP), `bs4` (Beautiful Soup, un Web Scraper), `ollama`, et `StreamLit`
- Faites la fonction `get_URL` qui prend en paramètre les mots-clés de la recherche et un nombre maximum de résultats
- Dans cette fonction, appelez la fonction `text` de `ddgs` et récupérez les URL ("`href`" dans une liste que vous renverrez à la fin de la fonction
- Dans le script `LLM.py`, appelez votre nouvelle fonction, et affichez dans le chat la liste des URL trouvées. Pour cela utilisez une `st.caption`

- f) Testez différentes recherches et vérifiez la pertinence des URL trouvées
- g) Dans votre script `Web.py`, faites la fonction `get_page_content` qui prend en paramètre une URL et le nombre maximum de caractères à renvoyer (pour ma part, j'ai mis 4000)
- h) Dans cette fonction, commencez par définir le header de votre requête. Il s'agit d'un dictionnaire qui doit contenir au minimum le champ `User-Agent`
- i) Appelez la fonction `get` de `httpx` avec en paramètres l'URL à lire et votre header. Vous pouvez aussi ajouter d'autres paramètres
- j) Si la réponse a le statut 200 (success), créez un objet de type `BeautifulSoup` sur le texte de résultat avec le paramètre `html.parser`
- k) Enlevez tous les éléments inutiles pour notre utilisation : `script`, `style`, `nav`, `header`, `footer`
- l) Récupérez le texte restant dans l'objet `BeautifulSoup` et transformez le en une chaîne de caractères unique dont vous renverrez les n premiers caractères selon le paramètre de votre fonction
- m) Ajoutez la fonction `web_query` qui prend en paramètre une liste d'URL et qui appelle dans une boucle la fonction que vous venez de créer pour former une chaîne de caractère unique qui est le contexte Web que vous renverrez
- n) Dans le script LLM, ajoutez l'appel à cette fonction juste après l'extraction des URL, et concatenez le contexte web renvoyé à votre contexte actuel
- o) Testez si votre LLM est désormais capable de répondre à des questions sur les pages web trouvées par le script `Web.py`
- p) Dans votre script `Web.py`, ajoutez la fonction `get_resume` qui prend une chaîne de caractères en entrée (potentiellement très longue) et qui renvoie un résumé de celle-ci. Pour ce faire, vous utiliserez un appel à `ollama.chat` (sans le streaming d'activé cette fois)
- q) Dans votre fonction `web_query`, ajoutez l'appel à la fonction de résumé pour la création de votre contexte web. Comme le résumé peut prendre un peu de temps (il s'agit d'une inférence), faites un ajout de `st.caption` pour afficher le nom de la page en cours de traitement (feedback important pour un utilisateur impatient!)

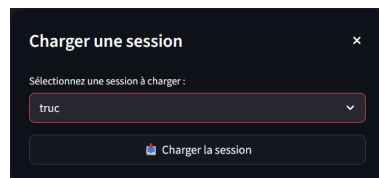
TP n° 5 : Les sessions

Dans ce TP, vous allez vous assurer de pouvoir sauvegarder vos sessions pour les réutiliser plus tard

- a) Avant de commencer à programmer les sessions, préparez l'interface graphique :



- b) Commencez par créer le dossier sessions qui vous servira de dossier de sauvegarde
- c) Créez le script `sessions.py` et importez les modules `json` (pour les sauvegardes), `streamlit` et `os`
- d) Faites la fonction `reset_session` qui supprime les messages que vous avez stockés dans `st.session_state`
- e) Appelez cette méthode dans le bouton "Nouvelle", et vérifiez que votre chat est bien reset quand vous cliquez sur le bouton
- f) Faites la méthode `save_session` qui construit un dictionnaire puis le sauvegarde au format JSON. Pour l'instant, ne mettez dans le dictionnaire que les messages échangés
- g) Faites la méthode `get_all_sessions` qui renvoie la liste des sessions sauvegardées. Enlevez l'extension ".json" pour que l'affichage soit un peu plus lisible
- h) Modifiez le comportement du bouton "Enregistrer" pour qu'il ouvre le popup suivant (la liste déroulante est remplie avec la liste des sessions)

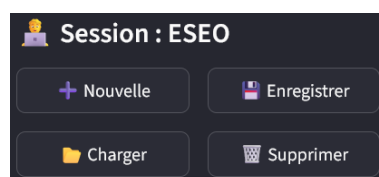


- i) Faites la méthode `load_session` qui charge dans un dictionnaire le contenu du fichier dont le nom est passé en paramètre (pensez à ajouter le ".json" que vous avez retiré juste avant), puis qui parcourt le dictionnaire pour copier le contenu de chaque clé de celui-ci dans `st.session_state`
- j) Ajoutez l'appel à cette fonction lors du clic sur le bouton "Charger la session"
- k) Faites plusieurs tests de chargement de session pour vérifier que les messages dans le chat change en fonction de la session
- l) Créez la fonction `delete_session` qui supprime la session courante. A la fin de cette fonction, pensez à appeler `reset_session`.
- m) Ajouter un appel à cette fonction lors du clic sur le bouton "Supprimer". Pensez à ajouter un popup, ça vous évitera de fausses manipulations

L'exploitation du `st.session_state`

StreamLit propose la variable globale `session_state` qui permet justement de stocker les informations d'une session (les cookies). C'est la variable parfaite pour stocker toutes nos variables de session

- a) Dans votre popup d'enregistrement de la session, renseignez le nom de la session dans une variable de session
- b) Modifier le titre de votre barre latérale pour afficher le nom de la session en cours (s'il existe)



- c) A la fin de votre fonction de barre latérale, stocker toutes les autres variables de votre session (en dehors des messages eux-mêmes) :

- LLM choisi
- Préprompt
- Corpus chois pour le RAG
- Les mots-clés Wikipédia
- La gestion des recherches sur le Web

- d) Modifiez votre méthode `save_session` pour prendre en compte les nouvelles variables à sauvegarder
- e) Faites des tests de sauvegarde et ouvrez les fichiers JSON pour vérifiez que toutes les variables sont bien présentes
- f) Modifiez également `reset_session`
- g) Reparcourez votre fonction de barre latérale, et pour chaque composant, ajoutez une condition : si la variable est renseignez dans le `session_state`, vous lui donnez cette valeur, sinon vous lui donnez sa valeur par défaut
- h) Faites une batterie de tests pour vérifier que tout fonctionne
- i) Reparcourez votre code, il y a plusieurs endroits où vous pouvez remplacer des paramètres de fonction par des variables de session
- j) Refaites une batterie de tests pour vérifier que tout fonctionne

Il ne vous reste plus qu'à tout mettre au propre pour l'examen !